Informe post-incidente — Simulacro de restauración de Food Store

TP Unidad 6 · Bases de Datos II · Matias Limina - Nicolas Monjelardi - Lautaro Aguero · PostgreSQL 17.11, `localhost:5432` · 28/10/2026

1. Respaldo utilizado

El último que se generó: `foodstore_20261028_181449.dump`. Es del **28/10/2026 a las 18:14:49**, pesa 3,7 MB, está en formato custom (`pg_dump -Fc`) y lo generó `backup_foodstore.ps1`. Junto con él quedaron los conteos de filas tomados justo antes de `pg_dump`.

2. Tiempo de restauración y validación

El simulacro empezó a las 18:57:38. El cronómetro arrancó inmediatamente antes de `createdb foodstore_restore_test` y paró cuando las validaciones terminaron bien.

Etapa	Tiempo
`createdb`	1,1 s
`pg_restore --no-owner --no-privileges`	11,5 s
Validaciones	0,4 s
Total (RTO observado)	**13,0 s**

Después, fuera del cronómetro, se ejecutó `ANALYZE` (3,1 s), porque `pg_restore` no restaura las estadísticas del planificador.

3. RPO y RTO

RPO: cuántos datos se pueden perder, medidos como la antigüedad del último respaldo. **RTO**: cuánto se tarda en volver a tener la base operativa y validada.

	Previsto (laboratorio)	Observado
RPO	24 h (un respaldo diario)	**42 min 49 s**
RTO	10 min	**13,0 s**

El RPO observado es bajo porque el respaldo era reciente, no porque el plan lo garantice. Con un respaldo diario, el peor caso se acerca a 24 h. Para bajarlo hacen falta respaldos más frecuentes o archivado de WAL. El RTO quedó muy por debajo del objetivo. Con 100 MB, el 88 % del tiempo lo usa `pg_restore` para recrear índices y restricciones.

4. Validaciones sobre `foodstore_restore_test`

Control	categoria	cliente	producto	pedido	pedido_detalle	Productos inválidos	Detalles inválidos
Antes del respaldo	3	20 003	50 005	200 001	621 853	0	0
Restaurado	3	20 003	50 005	200 001	621 853	0	0

7/7 OK. La consigna valida `detalle_pedido`; en el esquema la tabla es `pedido_detalle`. También coinciden los objetos de las dos bases: tablas, vistas, funciones, triggers, índices (12) y restricciones (16). `foodstore` no se tocó: tenía el mismo OID y los mismos conteos antes y después.

5. Problema real encontrado

La vista materializada de la base principal estaba desactualizada. En `foodstore_restore_test`, `mv_facturacion_categoria_mes` tiene 13 filas; en `foodstore`, 1. El respaldo no guarda su contenido: guarda un `REFRESH` que `pg_restore` ejecuta, así que la copia se recalculó con los datos actuales. En `foodstore` la vista se había creado antes de cargar los 200 000 pedidos y nunca se refrescó, así que sus reportes muestran datos viejos.

Otros problemas encontrados al probar los scripts:

- El respaldo y el chequeo de salud podían quedar colgados ante un bloqueo de otra sesión. Se les puso un límite de espera (`lock_timeout`, `--lock-wait-timeout`).
- Las tablas chicas nunca se analizaban: para `categoria`, el planificador estimaba 360 filas en lugar de 3.

6. Mejora concreta

Agregar al plan semanal, antes del respaldo, `REFRESH MATERIALIZED VIEW CONCURRENTLY mv_facturacion_categoria_mes`, que no bloquea las lecturas. El índice único que exige ya existe. También conviene sumar a la validación de la restauración la comparación del contenido de la vista. Así el respaldo y la base principal muestran lo mismo, y una diferencia como la de este simulacro se detecta en el control semanal y no en un incidente.

Para una base más grande, el siguiente ajuste de tiempo sería `pg_restore -j` (restauración en paralelo).

7. Trabajo asistido por IA y cómo se verificó

La consigna sugiere OpenCode. El equipo usó **Claude Code** en la terminal del repositorio para todo el TP: la SPEC, los cuatro scripts, las pruebas y la evidencia. Ningún resultado se aceptó sin ejecutarlo:

- **Pruebas manuales:** cada script se ejecutó a mano en sus casos de éxito y de error (base inexistente, sin credencial, bloqueos, respaldo truncado, conteos distintos).
- **Tests en bases descartables:** `test_maintenance.sh` sobre `foodstore_maint_test` y `test_restore.sh` sobre una base canario, escritos antes que los scripts. Se comprobaron con mutaciones: cambios deliberados en el código que el test debía detectar.
- **Revisión antes de cada commit:** un agente revisor de solo lectura revisó el diff.

Así se detectaron y corrigieron defectos de la IA, entre ellos:

- `$PSScriptRoot` vacío en PowerShell 5.1;
- acentos desfigurados en el log;
- chequeos que se colgaban detrás de un bloqueo;
- un control de destino que no distinguía mayúsculas;
- un test que usaba `foodstore` como destino de prueba.

El detalle está en `BITACORA.md` y `evidencias/`. No se versionan contraseñas: la clave sale de `pgpass.conf` y los clientes se invocan con `-w`.